

Firebase Security Rules

Firebase Security — Rules and Application-Layer Enforcement

This guide documents the access controls protecting CTP Job Cards data. Two layers are in play:

1. **Firebase Security Rules** — declarative rules enforced by Firestore and Cloud Storage.
2. **Application-layer enforcement** — checks inside `FirestoreService`, Cloud Functions, and role-gated UI.

⚠ **Important:** at time of writing, `firestore.rules` is **not** tracked in this repo. `firebase.json` only references `storage.rules` and `firestore.indexes.json`. Firestore rules are deployed but not version-controlled. Treat the production Firebase Console as the source of truth until `firestore.rules` is committed.

1. Storage rules

The only ruleset committed to the repo:

File: `storage.rules`

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /job_cards/{jobId}/{allPaths=**} {
      allow read: if true;
      allow write: if request.auth != null;
    }
  }
}
```

What this means

Path	Read	Write
<code>job_cards/{jobId}/**</code> (photos attached to job cards)	Public	Any authenticated user
Anything outside <code>job_cards/</code>	Denied (no matching rule)	Denied

Risk notes

- **Reads are open.** Any client with the bucket URL can read every job-card photo without auth. This is intentional for now (so notifications can preview thumbnails without re-auth), but means photos must not contain sensitive content that wouldn't be acceptable to leak.
- **Writes are any-authenticated.** Any signed-in user — operator, technician, manager — can write to any job's photo folder. There is no per-job permission check at the rule layer. If photo tampering becomes a concern, tighten this to require the writer's clock number to match the job's assigned or creator clock number.

Deployment

```
firebase deploy --only storage
```

2. Firestore rules (not in repo)

Because `firestore.rules` is not tracked, this section documents what the **deployed** rules need to enforce, sourced from the application code that depends on them.

Collections in use

Collection	Written by	Read by	Sensitivity
<code>job_cards</code>	<code>FirestoreService</code> , <code>onJobCardAssigned</code>	All authenticated users (filtered client-side by department/role)	Medium
<code>job_card_audit</code>	<code>FirestoreService.appendAudit()</code> only	Managers, Admin	High (audit log)
<code>employees</code>	Registration screen, Admin screen	All authenticated users (for routing/recipient lookup)	High (contains clockNo, position,

Collection	Written by	Read by	Sensitivity
			FCM tokens)
notifications	Cloud Functions only	Managers (Notification History), Admin	Medium (operational log)
copper_inventory / copper_transactions	Copper-authorized clock numbers (22 , 5421 , 20)	Copper-authorized clock numbers	Medium
notification_configs/global	Admin screen only	Cloud Functions (escalateNotifications)	High (controls who gets paged)
alertResponses	Notification action buttons (any auth user)	Cloud Functions (onAlertResponseCreated)	Low

Rules that should be in place

These are the access patterns the application **assumes** are enforced. If your deployed `firestore.rules` doesn't enforce them, the app still works but is open to abuse by a malicious authenticated user.

```
// Authenticated-only baseline
match /{document=**} {
  allow read, write: if request.auth != null;
}

// job_card_audit: append-only, no client deletes
match /job_card_audit/{auditId} {
  allow create: if request.auth != null;
  allow read: if request.auth != null;
  allow update, delete: if false;
}

// employees: only the user themselves or an admin can write
match /employees/{clockNo} {
  allow read: if request.auth != null;
  allow write: if isAdmin() || resource.data.uid == request.auth.uid;
}

// notification_configs: admin write, anyone read
match /notification_configs/{doc} {
```

```
allow read: if request.auth != null;
allow write: if isAdmin();
}
```

Where `isAdmin()` is determined by clock number lookup. **The current app has no admin claim** — the Admin screen is gated by a hardcoded password in client code (`home_screen.dart`), so a determined attacker could write directly to Firestore from a console. A proper fix is to set a Firebase custom claim `{ admin: true }` on admin accounts and check it in the rules.

Deployment

Until the rules are tracked:

1. Make changes via the Firebase Console (Firestore → Rules tab) or write a local `firestore.rules` and add it to `firebase.json`.
2. Deploy:

```
firebase deploy --only firestore:rules
```

3. **Commit the file once you've written it.** Untracked production rules are a recipe for accidental regression.

3. Application-layer enforcement

Rules are not the only line of defence. Several access controls live in the application:

`FirestoreService` audit logging

Every mutation to `job_cards` is mirrored as an append-only entry in `job_card_audit` (written from the same client). The audit trail captures `who`, `when`, `field`, and `before / after`. This is enforced by code path, not by rules — a client that bypassed `FirestoreService` could write to `job_cards` without auditing. The proper fix is a Firestore trigger that writes the audit entry server-side from `onJobCardUpdated`.

Role-based UI gating

Defined in `lib/utils/role.dart` — see also [CLAUDE.md](#). Roles inferred from `Employee.position`:

Role	Gated screens
Technician	My Assigned Jobs visibility

Role	Gated screens
Manager	Dashboard tab, Daily Review (web), Notification History
Operator	Default; no special gating
Admin	Settings → Admin (password-gated), AdminScreen, GeofenceEditor

These are **UI gates only**. A malicious client could still call Firestore directly and bypass them — which is why the corresponding rules above must be deployed.

Admin password gate

`home_screen.dart` — the Admin button in Settings prompts for a hardcoded password (search `correctPassword`). This is convenience, not security. Anyone with the password (or with the APK source) has admin UI access. **Tighten with Firebase custom claims** for any deployment beyond the current trusted user base.

Cloud Functions auth context

Callable functions check `context.auth` before performing sensitive operations:

- `migrateJobStatuses` requires `context.auth` (authenticated)
- `createCustomToken`, `clearEscalationStamps` — currently open to any caller; tighten if exposure becomes a concern

4. Testing rule changes

Use the Firebase Emulator Suite:

```
firebase emulators:start --only firestore,storage
```

With `firestore.rules` and `storage.rules` in the repo and `firebase.json` updated to point to both:

```
"firestore": {
  "rules": "firestore.rules",
  "indexes": "firestore.indexes.json"
},
"storage": {
  "rules": "storage.rules"
}
```

The emulator UI at `http://localhost:4000` includes a Rules Playground for ad-hoc rule testing without redeploying.

Write rule unit tests with `@firebase/rules-unit-testing`:

```
npm install --save-dev @firebase/rules-unit-testing
```

See the Firebase docs for the test harness pattern.

5. Action items (post-rollout)

1. **Track `firestore.rules` in the repo** — copy the deployed rules out of the Firebase Console and commit them. Update `firebase.json` to reference the file.
2. **Replace the hardcoded admin password with a Firebase custom claim** — set `{ admin: true }` on admin accounts via the Admin SDK, then check `request.auth.token.admin == true` in rules.
3. **Move `job_card_audit` writes server-side** — a Firestore trigger ensures every mutation is audited, even if a client bypasses `FirestoreService`.
4. **Tighten storage writes** — require the writer's clock number to be on the job's assigned or creator list before allowing photo uploads to that job's folder.